

מבוא למחשוב ענן - סמסטר חורף התשפו

תרגיל בית 3 - עבודה בצוותי העבודה

צוות Chameleon

https://colab.research.google.com/drive/12uA5krh7UgzXWNQstiTZ4WPhte-AI3kj	קישור למחברת ההגשה:
https://github.com/SteveTrabajo/Cloud-Computing	קישור לתיקיית גיט:

שימו לב: למטלה זו שלושה חלקים

חלק ראשון (10 נקודות).

עליכם להגדיר את בעלי התפקידים לאיטרציה זו (יש להחליף מתרגיל בית 2) יש לרשום מי מהנדסת. המערכת, האחראי. ת בתרגיל זה.

מהנדס מערכת: נתן נימץ

שם חבר הצוות	משימות שהוקצו	משימות שהושלמו	Acceptance test – בדיקת קבלה
ניקולאי סבצנקו	מסמך WORD, QA, HF, INTEGRATION	הכל	Acceptance test: בדיקת העלאת תמונת עלה במסך Leaf Check וקבלת תוצאת הערכה תקינה מבוססת מודל הראייה של HuggingFace
נתן נימץ	תכנון, סיום מסכים, הוספת gamification, chatbot, הוספת אג'נט	הכל	Acceptance test: בדיקת צבירת נקודות ועדכון תגים במסך Achievements לאחר סימון השקיה, ובדיקת קבלת מענה רלוונטי מבוסס הקשר במסך ה Assistant בנוסף בדיקת האייג'נט

חלק שני: בניית המערכת (70 נקודות)

1. כל הפיצ'רים שהתחלתם לממש בתרגיל בית 2 (35 נקודות) :
כן.

2. שימוש ב- microservices (תרגול 8) - עליכם לעשות שימוש לפחות בשניים. פרטו באילו שירותים עשיתם שימוש, ומה היתרון בשימוש בשירותים אלו. (10 נקודות)

המערכת מבוססת על מספר שירותים חיצוניים עצמאיים שכל אחד מהם אחראי על תחום אחד, במקום שרת מונוליטי אחד.

השתמשנו לפחות בשניים:

Firestore Realtime Database - שירות ענן לאחסון נתונים.

שומר את אינדקס החיפוש (plant_disease_index), את קריאות החיישנים (sensor_readings) ואת נתוני המשחק (gamification).

הגישה מתבצעת דרך קריאות REST פשוטות בלי צורך ב SDK.

GEMINI LLM של Google - שירות ה AI שאחראי על כל היכולת הלשונית: מנוע ה RAG, הצ'אטבוט, וניתוח תמונת העלה.

הגישה דרך REST ל generativelanguage.googleapis.com, כולל תמיכה ב streaming.

בנוסף השתמשנו ב **MQTT broker** ציבורי (HiveMQ) לפרסום קריאות חיישנים בזמן אמת, בשרת חיישנים מבוסס ענן שממנו נשלפים ערכי הטמפרטורה, הלחות והקרקע, וב HuggingFace Datasets לשליפת דאטסט עלים לצורך השוואת מצב הצמח.

היתרון בשימוש ב microservices:

הפרדת אחריות - כל שירות עושה דבר אחד וניתן להחליף אותו בלי לגעת בשאר,

חוסן - אם שרת החיישנים לא זמין, המערכת נופלת בחן ל-fallback מקומי ולא קורסת,

וסקיילביליות - כל שירות מתוחזק ומתרחב בנפרד על ידי ספק הענן שלו, בלי שנצטרך לתחזק תשתית משלנו

3. מהם ה KPI-ה הרלוונטיים בפרויקט שלכם? הסבירו. (10 נקודות)

המדדים שבחרנו בוחנים גם את בריאות הצמח וגם את איכות המערכת:

אחוז זמן בטווח בריא: אחוז הקריאות שבהן כל הערכים, כמו טמפרטורה, לחות, לחות קרקע ורטיבות עלה, נמצאים בטווח התקין שהגדרנו.

זהו המדד המרכזי כי הוא בוחן את המטרה האמיתית של המערכת: לשמור על צמח בריא.

כיסוי דגימות: מספר הקריאות שנאספו לאורך זמן ורציפותן.

מדד זה בוחן את אמינות איסוף הנתונים, כאשר פערים גדולים מעידים על נפילות של החיישן או השרת.

ביסוס החיפוש: אחוז השאלות שנענו על בסיס המאמרים המאונדקסים לעומת ידע כללי של המודל. המדד בוחן עד כמה מנוע ה-RAG באמת נשען על המקורות ולא סתם ממציא.

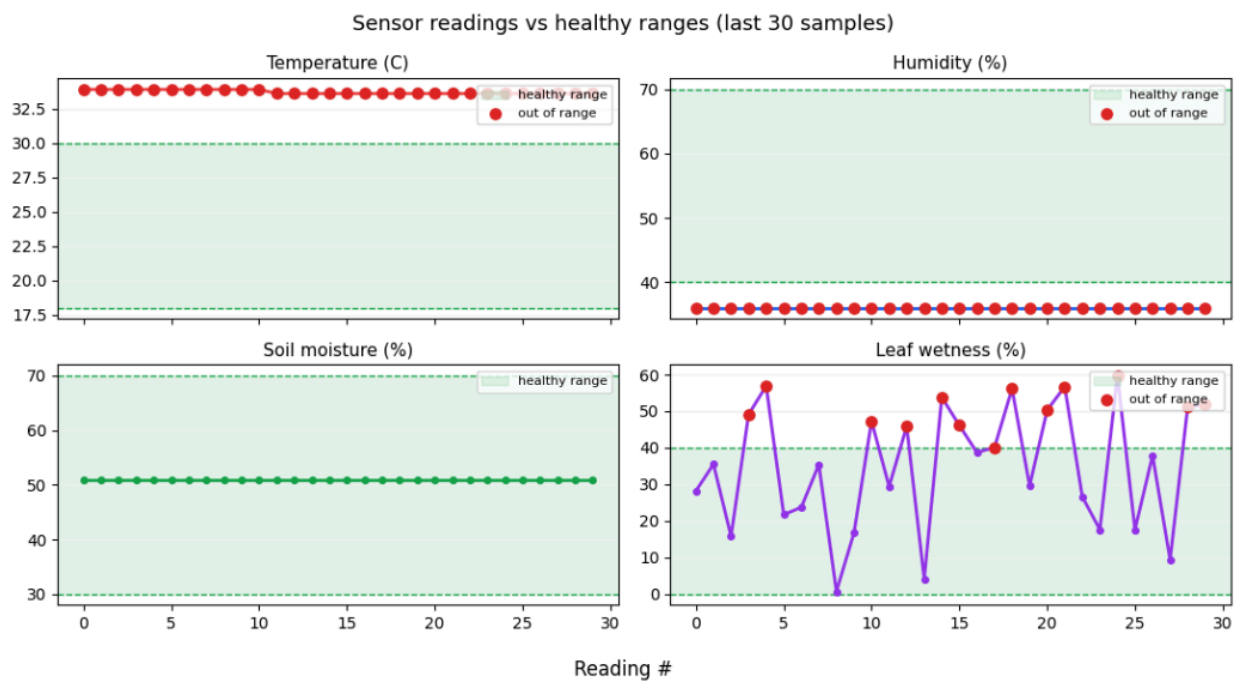
עמידה בהשקיה: כמות ההשקיות שבוצעו בזמן לעומת השקיות שבוצעו באיחור.
מדד זה בוחן את שיפור ההתנהגות של המשתמש בזכות המערכת.

מעורבות משתמש: כמות הנקודות ומספר הפעולות שנצברו דרך מנגנון המשחק.
מדד זה בוחן כמה המשתמש חוזר ומשתמש במערכת.

4. מידע עתק: השתמשו במודלים שהוצגו בשבוע 9 ובצעו ניתוח על המידע שהגיע מהחיישנים ושמרתם ב DB. הציגו בוורד גרף נתונים של חיתוך לבחירתכם (5 נקודות).

5.

הניתוח שבחרנו הוא **חיתוך לפי זמן**: מגמת הטמפרטורה, הלחות, לחות הקרקע ורטיבות העלה לאורך הקריאות האחרונות, עם סימון טווחי הבריאות.



6. אתגרים שעלו במהלך העבודה, וכיצד התמודדתם איתם (לדוגמא - DB גדול מדי, יצירת ויזואליזציה, מבנה הקוד וכו'). (5 נקודות).

ריכוז כל המסכים בתא אחד

בתרגיל הקודם כל פיצ'ר היה תא נפרד.

איחוד הכל לאפליקציה אחת דרש לבנות מבנה טאבים ב ipywidgets עם מצב משותף (SENSOR_LOG, GAME) בין המסכים.

פתרנו עם פונקצייה שבונה כל מסך בנפרד ובמקרה כשל מציגה הודעה במקום להפיל את כל האפליקציה.

חוסר יציבות של שירותים חיצוניים

שרת החיישנים על Render ישן לפעמים, וה broker של MQTT לא תמיד זמין.

הוספנו בכל קריאת רשת בלוק try/except עם fallback, כך שהמערכת ממשיכה לעבוד גם כשירות חיצוני נופל.

רינדור גרפים בתוך טאב

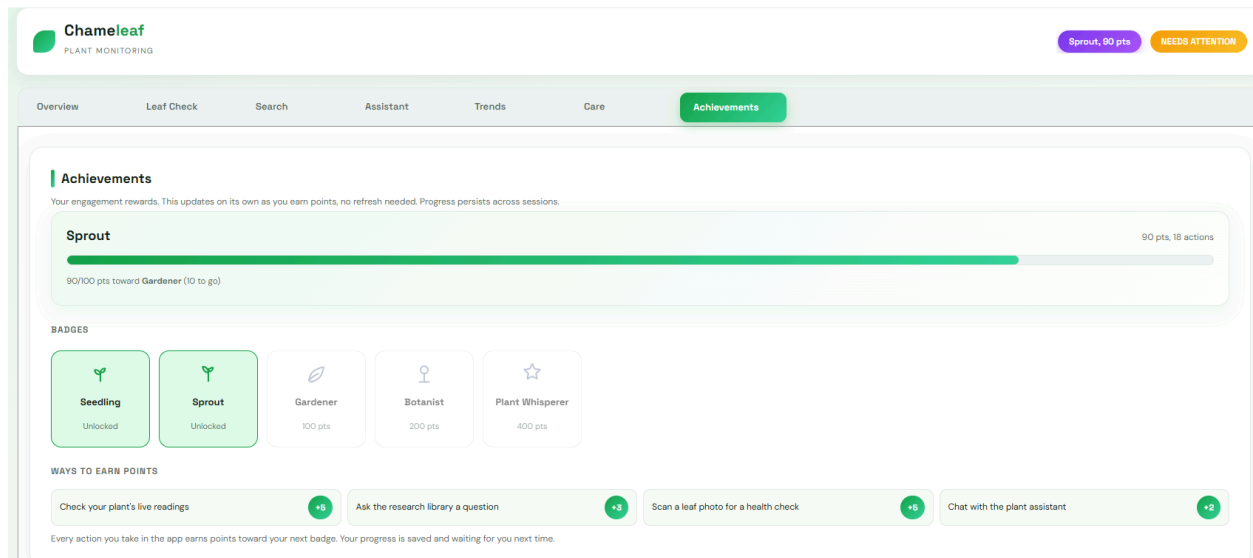
גרף matplotlib בתוך Output נטה להיעלם בזמן מעבר בין טאבים. פתרנו בהמרת הגרף ל PNG בקידוד base64 והצגתו כ `<img>` בתוך widget של HTML, מה שמרנדר אמין תמיד.

ריבוי נתונים בטבלת הקריאות

כשמצטברות הרבה קריאות, טבלה אחת ארוכה הפכה ללא קריאה. הוספנו pagination וסינון לפי תאריך במסך Trends. זמן טעינה של מודל התמונה. אתחול CLIP והדאטהסט של HF לוקח כדקה. חיממנו אותו מראש (`hf_init_` בתא 23) עוד לפני עליית האפליקציה, כדי שההעלאה של המשתמש תהיה מיידית.

7. יש להוסיף ממשק כלשהו של משחק כרצונכם (אלמנט בודד כגון צבירת נקודות, תגים, שלבים וכו). (5 נקודות).

מסך המשחק:



8. **בונוס** - ממשו לוגיקת סוכן לצורך כלשהו של המערכת. הסבירו כיצד מתבטא בקוד (5 נקודות).

האייגנט הוסף לאחר הקלטת הוידאו

הסוכן עובד בתוך הפונקציה `agent_run`.

הוא שולח ל Gemini הנחיית `system prompt` שמפרטת את הכלים הזמינים בתוך `AGENT_TOOLS` ודורש לקבל תשובה בפורמט JSON

בכל איטרציה של הלולאה הוא קורא את ה `JSON` בעזרת `parse_action_`, מוצא את הכלי המבוקש במילון `AGENT_TOOLS`, מריץ אותו, ומוסיף את התוצאה ל `transcript`. התהליך חוזר על עצמו עד שהמודל מחזיר הודעת סיום בצורה של `{...:"final"}` וזה מוגבל למקסימום 5 שלבים.

הכלים במערכת הם פונקציות רגילות לגמרי כמו קריאת חיישנים, חיפוש RAG מתוך מאמרים וכלי שבאמת משנה את תדירות ההשקיה. בצורה הזו המודל בוחר את השלבים בעצמו ויכול לבצע פעולות אמיתיות ולא רק להחזיר תשובות טקסט.

חלק שלישי: סגירת הפרויקט (20 נקודות)

הכינו מסמכים המתארים את הפרויקט שלכם:

- יש לבנות תיק למתכנת הכולל את שמות כל הקבצים המרכזיים, פונקציות מרכזיות, קטעי קוד/תבניות עיצוב מעניינים שהשתמשתם בהם.

המערכת כולה מיושמת במחברת אחודה, המחולקת לוגית לנושאים הבאים:

- הורדת מאמרים ובניית אינדקס: שלבי הכנת הנתונים ותשתית החיפוש.
- הגדרת מנוע השפה ואחזור המידע: ארכיטקטורת הבינה המלאכותית והזנת ההקשר.
- שכבת החיישנים, פרוטוקול התקשורת ובסיס הנתונים: ניהול התקשורת, העברת ההודעות ואיסוף הנתונים בזמן אמת.
- רכיב המשחוק: מנגנון הגברת מעורבות המשתמשים על ידי נקודות ופעולות.
- חיפוש תמונה: עיבוד ויזואלי וחיפוש שכן קרוב במאגר נתונים חיצוני.
- האפליקציה המלאה: רינדור הממשק, ניהול המצב וטעינת כלל המסכים.

פונקציות מרכזיות במערכת

- `build_index`: מקבלת את מאמרי המקור ובונה אינדקס הפוך על סמך מונחים נבחרים לטובת שליפה מהירה.
- `rag_context | search`: מנהלות את אחזור המסמכים הרלוונטיים לשאליתה ובניית ההקשר המדויק עבור מודל השפה.
- `gemini`: מבצעת קריאה מרוכזת למודל, תוך תמיכה מלאה בהיסטוריית שיחה ובעיבוד קלטי תמונה.
- `load_history | read_sensors`: אחראיות על שליפת נתוני חיישנים חיוביים בזמן אמת והיסטוריית קריאות, כולל מנגנון גיבוי מובנה במקרה של תקלה.
- `hf_image_search`: מייצרת ייצוג וקטורי של תמונה בעזרת מודל ייעודי ומבצעת חיפוש שכן קרוב בדאטהסט הקיים.
- `award`: מנוע הניקוד האחראי על הענקת נקודות, מעקב אחר התקדמות ועדכון תגים למשתמש.
- פונקציות ניהול המסכים: סדרת פונקציות ייעודיות לבניית הטאבים השונים בממשק, הכוללת את

מסך הבית, העלאת תמונות, חיפוש, צ'אט, מגמות, טיפול בצמח והישגים.

ארכיטקטורה ותבניות עיצוב בולטות

- אינדקס הפוך מתקדם: שימוש ברשימות מיקומים ותדירות מילים לצורך דירוג אופטימלי ומדויק של תוצאות החיפוש.
- ארכיטקטורת אחזור מבוססת מקורות: מנגנון המזין הקשר עשיר מתוך המאמרים למודל השפה, לצד הנחיית מערכת קשיחה לציטוט המקורות המדויקים ומניעת הזיות.
- תבנית הנדסת חוסן: הגדרת גרסאות דמה עבור כל תלות חיצונית או רכיב חומרה בראש קוד האפליקציה, המבטיחה רינדור רציף ועבודה חלקה של הממשק גם במקרה של ניתוק או שגיאת תקשורת.
- אופטימיזציית ויזואליזציה: המרת גרפים ומפות לפורמט מחרוזת נתונים המאפשרת רינדור מהיר, יציב ואמין בתוך הטאבים השונים של אפליקציית הווב.
- ניהול תהליכים ברקע: שימוש בתהליכון רקע ייעודי לרענון נתונים אוטומטי, המנוהל על ידי אירוע עצירה מובנה לסגירה נקייה ובטוחה של המשאבים בזמן סגירת היישום.

2. יש לבנות תיק למשתמש, הכולל הסבר כללי על המערכת, פירוט מסכים, מעברים בין מסכים והסבר על טעויות אפשריות.

תיאור מערכת Chameleaf וארכיטקטורת הממשק

הסבר כללי

מערכת Chameleaf היא פלטפורמה מתקדמת לניטור בזמן אמת של צמח המונסטרה (*Monstera deliciosa*). המערכת אוספת נתונים מרשת חיישנים, מציגה את מצבו העדכני של הצמח, מספקת תשובות מבוססות מחקר אקדמי לטיפול במחלות, ומאפשרת לבחון את בריאות העלים באמצעות כלי בינה מלאכותית על בסיס ניתוח תמונה. בנוסף, המערכת משלבת מנגנון משחוק שמטרתו לעודד ולהתגמל את המשתמש על מעורבותו ועל טיפול עקבי בצמח.

מבנה הממשק וניהול המסכים

האפליקציה בנויה במבנה של לשוניות, כאשר המעבר בין המסכים השונים מתבצע בלחיצה ישירה על הלשונית המתאימה. תשתית המערכת מבוססת על מצב משותף, המבטיח כי נתונים כגון קריאות החיישנים ונקודות הזכות של המשתמש יתעדכנו ויסתנכרו באופן מלא ורציף בכל הלשוניות במקביל.

- מסך Overview: מציג את סטטוס הצמח על סמך קריאת החיישנים האחרונה שהתקבלה. המסך כולל כפתור ייעודי לביצוע דגימה מיידית ואפשרות להפעלת מנגנון רענון נתונים אוטומטי.
- מסך Leaf Check: מאפשר למשתמש להעלות תמונה של עלי הצמח לצורך ניתוח ובדיקת מצב בריאותם באמצעות מודל בינה מלאכותית, תוך השוואה לספריית עלים מובנית.

- **Search:** מיועד להפניית שאלות בנושא הטיפול בצמח. מנוע החיפוש משלב בין מאמרי מחקר אקדמיים לבין ידע מומחה, ומציג את המקורות המדויקים ששימשו לניסוח התשובה.
- **Assistant:** צ'אטבוט אינטראקטיבי המאפשר למשתמש לנהל שיחה חופשית על דרכי הטיפול בצמח ועל נתוני החיישנים החיים.
- **Trends:** מציג טבלה וגרף ויזואלי של קריאות החיישנים לאורך זמן. המסך כולל כלי סינון מתקדמים לפי טווחי תאריכים וכמויות נתונים, לצד מנגנון דפדוף מובנה.
- **Care** מאפשר הגדרה של מרווחי השקיה, סימון ביצוע השקיה בפועל, והצגת התראות ייעודיות במקרה של זיהוי קרקע יבשה.
- **Achievements:** מרכז את רכיבי המשחק של המערכת, לרבות צבירת נקודות זכות, קבלת תגים ומעבר בין רמות שונות.

ניהול שגיאות והתמודדות עם תקלות

המערכת תוכננה עם דגש רב על חוסן הנדסי ועמידות מפני קריסות. במקרה שבו שירות חיצוני אינו זמין - בין אם מדובר בשרת החיישנים, בפרוטוקול התקשורת MQTT או במודל השפה של Gemini - המערכת תציג ערך חלופי מוגדר מראש או הודעה מתאימה במקום להפסיק את פעולתה.

במצב שבו רינדור של מסך מסוים נכשל לחלוטין, תוצג למשתמש הנחיה ברורה המבקשת להריץ מחדש את כל תאי המחברת. כמו כן, אם מנוע החיפוש אינו מצליח לאתר מאמר אקדמי תואם לשאילתה, המערכת תציין זאת במפורש בפני המשתמש ותבהיר כי התשובה המוצגת מבוססת על הנחיה כללית בלבד ולא על מקור מחקרי מאונדקס.

שקיפות אלגוריתמית וניהול נתונים

ארכיטקטורת המערכת דוגלת בשקיפות מלאה מול המשתמש לגבי אופן איסוף הנתונים וכיצד מתקבלות ההחלטות:

- במסך החיפוש: כל תשובה מסומנת באופן ברור באמצעות תג ייעודי המציין האם היא מבוססת על מאמרי מחקר ובינה מלאכותית או על מומחה צמחים ובינה מלאכותית. לצד זאת, מוצגת רשימת המקורות המלאה כדי לאפשר למשתמש לדעת בדיוק על מה נשענת התשובה.
- במסך בדיקת העלה: המערכת מבהירה למשתמש מראש כי הבדיקה מהווה הערכה בלבד של מודל הבינה המלאכותית בשילוב ספריית העלים, וכי אין התחייבות לדיוק מדעי מוחלט.
- במסך המבט הכללי: קיים הסבר מפורט המציג כיצד מחושב סטטוס הצמח על ידי השוואה ישירה של הקריאות לטווחי בריאות שהוגדרו מראש.
- שמירת הנתונים: כל הנתונים הנאספים מהחיישנים, הכוללים טמפרטורה, לחות, לחות קרקע ורטיבות עלה, גלויים לחלוטין למשתמש בתוך טבלת המגמות, ונשמרים בצורה מסודרת בבסיס הנתונים Firebase לצד נתוני המשחק ומערכת הניקוד.

3. התייחסו בתיק המשתמש לשקיפות אלגוריתמית – כיצד הבהרתם למשתמשים את האלגוריתמים והנתונים הנאספים בקוד שלכם? - **מפורט בתוך התיק משתמש**

קישור לסרטון של המערכת